

## שאלה 4 - שיפור ביצועים: פתרון

### 4. שיפור ביצועים (10%)

השאלתה הבאה נכתבה לבסיס הנתונים IMDB מהקורס. מטרת השאלתה היא לשלוף רשימת הבמאים העצובים, כאלו שמעולם לא ביימו קומדיה.

```
select *
from
directors as d
join
movies_directors as md
on
d.id = md.director_id
where
md.movie_id not in
( select distinct movie_id
from
movies_genres
where
genre = 'Comedy'
order by
movie_id
)
;
```

הציעו לפחות 3 תיקונים ושיפורים לשאלתה. אתם יכולים ליצור טבלאות נוספות ולהגדיר אינדקסים במידה שיועילו לכם.

1. עבור כל שינוי  
a. ציינו מה הבעיה שהוא מתקן.  
b. ציינו חסרונות אם יש.
2. כיתבו את השאלתה המתוקנת.

## תיקונים ושיפורים אפשריים

### 1. בחירת עמודות ספציפיות במקום שימוש ב- **SELECT \***

- **מה הבעיה שהוא מתקן:** השליפה המקורית משכה את כל העמודות מכל הטבלאות, מה שמייצר עומס מיותר על זיכרון השרת ורוחב הפס. הגדרת העמודות הרלוונטיות בלבד (**d.id**), (**d.first\_name**, **d.last\_name**) מקטינה משמעותית את כמות הנתונים המועברת ומאפשרת ביצועים מהירים יותר.
- הערה: בחירת העמודות הרלוונטיות נתונה לדיון ושיקול דעת. במידה ו

## 2. ביטול ה-JOIN לטבלת `directors` במידה ונדרש רק מזהה הבמאי (`director_id`)

- **מה הבעיה שהוא מתקן:** השאילתה המקורית מבצעת JOIN בין טבלת `directors` לטבלת `movies_directors`. פעולת הקישור הזו צורכת משאבי זיכרון וזמן עיבוד. אם מטרת השאילתה (או הדרישה העסקית) מסתפקת בשליפת המזהה המספרי של הבמאים העצובים, ניתן לוותר לחלוטין על הקריאה מטבלת `directors`. טבלת `movies_directors` מכילה גם את ה-`director_id` וגם את ה-`movie_id`, ולכן ניתן לבצע את הסינון (Anti-Join) מול הקומדיות (ישירות עליה). ביטול ה-JOIN חוסך קריאה וסריקה של טבלה שלמה ומקצר את זמן הריצה משמעותית.

### • חסרונות אם יש:

- מאבדים את הגישה לפרטים האישיים של הבמאי (כגון שם פרטי ושם משפחה, שאותם ניתן לשלוף רק מטבלת `directors`), ולכן ייעול זה ישים רק כאשר נדרשת רשימת מזהים בלבד.
- שינוי לוגי בקצה: במאי שרשום במסד הנתונים אך מעולם לא ביים אף סרט (ולכן אינו מופיע כלל בטבלת `movies_directors`), לא יופיע בתוצאות השאילתה, למרות שטכנית הוא עונה על התנאי "מעולם לא ביים קומדיה".

## 3. ביטול פעולת מיון מיותרת (`ORDER BY movie_id`)

- **מה הבעיה שהוא מתקן:** בשאילתה המקורית הוגדר מיון בתוך תת-השאילתה. לפעולת מיון יש עלות חישובית גבוהה, אך אין לה שום משמעות לוגית או השפעה על התוצאה כאשר היא מתבצעת בתוך תנאי או טבלה זמנית. הסרתה חוסכת ממנוע ה-SQL פעולת מיון יקרה לחינם, ומקצרת את זמן העיבוד

## 4. ביטול פעולת `DISTINCT` מיותרת בתוך תת-השאילתה

- **מה הבעיה שהוא מתקן:** בשאילתה המקורית נעשה שימוש ב-`DISTINCT` בתוך תת-שאילתה שמשמשת לתנאי `NOT IN`. האופרטורים `IN` ו-`NOT IN` בודקים אך ורק קיום או היעדרות של ערך (Membership), ולכן אין שום חשיבות לכפילויות בתוך הקבוצה. גם במקרה של שיפור השאילתה והמרה ל-`LEFT JOIN` מול טבלת הז'אנרים (לפי `movie_id`), ה-`DISTINCT` נותר מיותר ופוגע בביצועים: מכיוון שכל סרט מקושר לז'אנר 'Comedy' לכל היותר פעם אחת, אין סכנה להכפלת שורות (Row Explosion) בעת החיבור. פקודת ה-`DISTINCT` מאלצת את מסד הנתונים לבצע פעולות מיון או גיבוב (Sort/Hash) יקרות לחלוטין לחינם, רק כדי להסיר כפילויות שלא היו משפיעות על התוצאה במילא. הסרתה חוסכת משאבי עיבוד (CPU) וזיכרון.

## 5. שיפור מבני מתקדם: יצירת תת-טבלה מראש עם אינדקס ייחודי

- **מה הבעיה שהוא מתקן:** במקום לחשב את רשימת "במאי הקומדיות" מחדש בכל ריצה של השאילתה דרך חיבור טבלאות כבד, ניתן לייצר מראש טבלה פיזית קטנה (או Materialized View) שתכיל אך ורק את מזהי הבמאים הללו. על טבלה זו ניתן להגדיר אינדקס ייחודי (**UNIQUE INDEX**). מכיוון שמדובר בקבוצת נתונים קטנה בהרבה, חיפוש ה-**LEFT JOIN** מולה יהיה מהיר יותר.
- **חסרונות אם יש:** פתרון זה צורך שטח אחסון נוסף במסד הנתונים ודורש תחזוקה (הקמת מנגנון, כגון טריגרים או תהליך אצווה, שיעדכן את הטבלה הייעודית בכל פעם שמתווסף סרט קומדיה חדש או משתנה ז'אנר).

## 6. שימוש ב-DISTINCT בייצור רשימת במאי הקומדיות

- **מה הבעיה שהוא מתקן:** במאי עשוי לביים מספר קומדיות במהלך הקריירה שלו. ללא **DISTINCT**, ה-**director\_id** שלו יופיע ב"רשימה השחורה" מספר פעמים כמספר הקומדיות שביים. השימוש ב-DISTINCT מבטיח שכל במאי יופיע פעם אחת בלבד, מה שמקטין את גודל קבוצת הנתונים בזיכרון ומונע סריקות כפולות ומיותרות בשלב ה-**JOIN** המאוחר יותר.
- **חסרונות אם יש:** פקודת **DISTINCT** מאלצת את מסד הנתונים לבצע פעולת מיון או גיבוב (Hash/Sort) פנימית כדי להסיר כפילויות, מה שצורך מעט משאבי עיבוד (CPU) בתחילת ריצת השאילתה.

## 7. שימוש ב-LEFT JOIN (תבנית Anti-Join) במקום NOT IN לייעול הביצועים

- **מה הבעיה שהוא מתקן:** פקודת **NOT IN** המופעלת על תת-שאילתה (במיוחד ב-MYSQL) נוטה להיות כבדה, איטית ורגישה מאוד לערכי **NULL** (אם מוחזר **NULL** אחד בתת-השאילתה, התנאי כולו קורס ומחזיר 0 תוצאות). ההמרה ל-**LEFT JOIN** מול ה-CTE יחד עם תנאי **IS NULL** (תבנית ה-Anti-Join) מנצלת את מנגנון הקישור היעיל של מנוע המסד. זה מקצר משמעותית את זמן הריצה (Execution Time) ומבטיח טיפול נכון בערכים חסרים.

## 8. שימוש ב-CTE (פקודת WITH) במקום תת-שאילתה

- **מה הבעיה שהוא מתקן:** השאילתה המקורית מקננת את הלוגיקה בתוך תנאי ה-**WHERE**. שימוש ב-CTE מאפשר להגדיר את "הרשימה השחורה" של במאי הקומדיות כבלוק קוד נפרד ועצמאי בראש השאילתה. זה משפר משמעותית את קריאות הקוד (Readability), מקל על תחזוקה ודיבוג, ומאפשר חלוקה לוגית ברורה של שלבי העיבוד.
- **הערה:** אמנם השימוש ב-CTE משפר את איכות הקוד, אבל לא בהכרח משפר ביצועים.

## 9. תיקון הלוגיקה מסינון ברמת הסרט לסינון ברמת הבמאי

- **מה הבעיה שהוא מתקן:** השאלתה המקורית מכילה כשל לוגי – היא סורקת ברמת הסרט במקום ברמת הבמאי. כך, במאי שביים קומדיה אך גם סרט מדרמה, יוחזר בתוצאות בגלל הדרמה. התיקון משנה את הגישה: תחילה מאתרים את המזהים (`director_id`) של כל מי שביים קומדיה, ולאחר מכן מסננים אותם החוצה מתוך רשימת הבמאים הכוללת.
- **חסרונות אם יש:** דורש שינוי מהותי במבנה השאלתה ומעבר מחיפוש פשוט של `movie_id` לעבודה מול רשימות מקושרות של במאים.

שאלתות מתוקנת (קיימות מספר אופציות בהתאם לשיפורים שנבחרו)

### ללא תיקון הכשל הלוגי:

-- שיפור 1+2: שימוש ב-CTE ויצירת רשימת קומדיות (ללא DISTINCT וללא ORDER BY מיותרים)

WITH comedy\_movies AS (

SELECT movie\_id

FROM movies\_genres

WHERE genre = 'Comedy')

-- שיפור 3: צמצום שליפת הנתונים לעמודות הרלוונטיות בלבד במקום \* SELECT

SELECT d.id, d.first\_name, d.last\_name

FROM directors AS d

-- שיפור 4: כתיבה מפורשת של (Best Practice INNER JOIN)

INNER JOIN movies\_directors AS md

ON d.id = md.director\_id

-- שיפור 5+6: שימוש ב-LEFT JOIN כתבנית Anti-Join במקום NOT IN האיטי

LEFT JOIN comedy\_movies AS cm

ON md.movie\_id = cm.movie\_id

-- סינון השורות כך שנישאר רק עם סרטים שלא הוגדרו כקומדיה

WHERE cm.movie\_id IS NULL;

## עם תיקון הכשל הלוגי דרך שימוש בCTE:

```
with comedy_directors as (  
  select distinct md.director_id  
    from movies_directors as md  
  join movies_genres AS mg  
    on md.movie_id = mg.movie_id  
  where mg.genre = 'Comedy'  
)  
select d.id, d.first_name, d.last_name  
  from directors as d  
 left join comedy_directors as cd  
    on d.id = cd.director_id  
 where cd.director_id is null;
```